

程序设计训练

OJ 在线评测系统

—— 课程大作业实验报告 ——

作业模块：基础模块 Step 1–6（30 分）+ 进阶模块
AI 智能命题（10 分）

技术栈：FastAPI（异步）+ Streamlit + JSON 文
件存储 + subprocess/psutil 评测

代码规模：后端 11 个模块 2013 行 + 前端 605 行
= 2618 行

git 提交：9 次 Conventional Commits 规范提交

报告生成于 2026 年 9 月

目录

1. 系统功能与设计
2. 关键实现与难点
3. 成果展示
4. AI 使用说明
5. 总结与建议

1. 系统功能与设计

1.1 系统目标

本系统实现了一个轻量级但功能完整的 **OJ (Online Judge) 在线评测系统**，覆盖题目管理、代码评测、用户与权限、评测日志、Streamlit 前端、**AI 智能命题**六大模块，**所有后端 API 均使用 FastAPI 异步接口 (`async def`)**，符合"不使用异步编程的无法拿到本次作业分数"的硬性要求。

1.2 系统架构

系统采用经典的 **前后端分离 + JSON 文件存储架构**，自上而下分为四层：

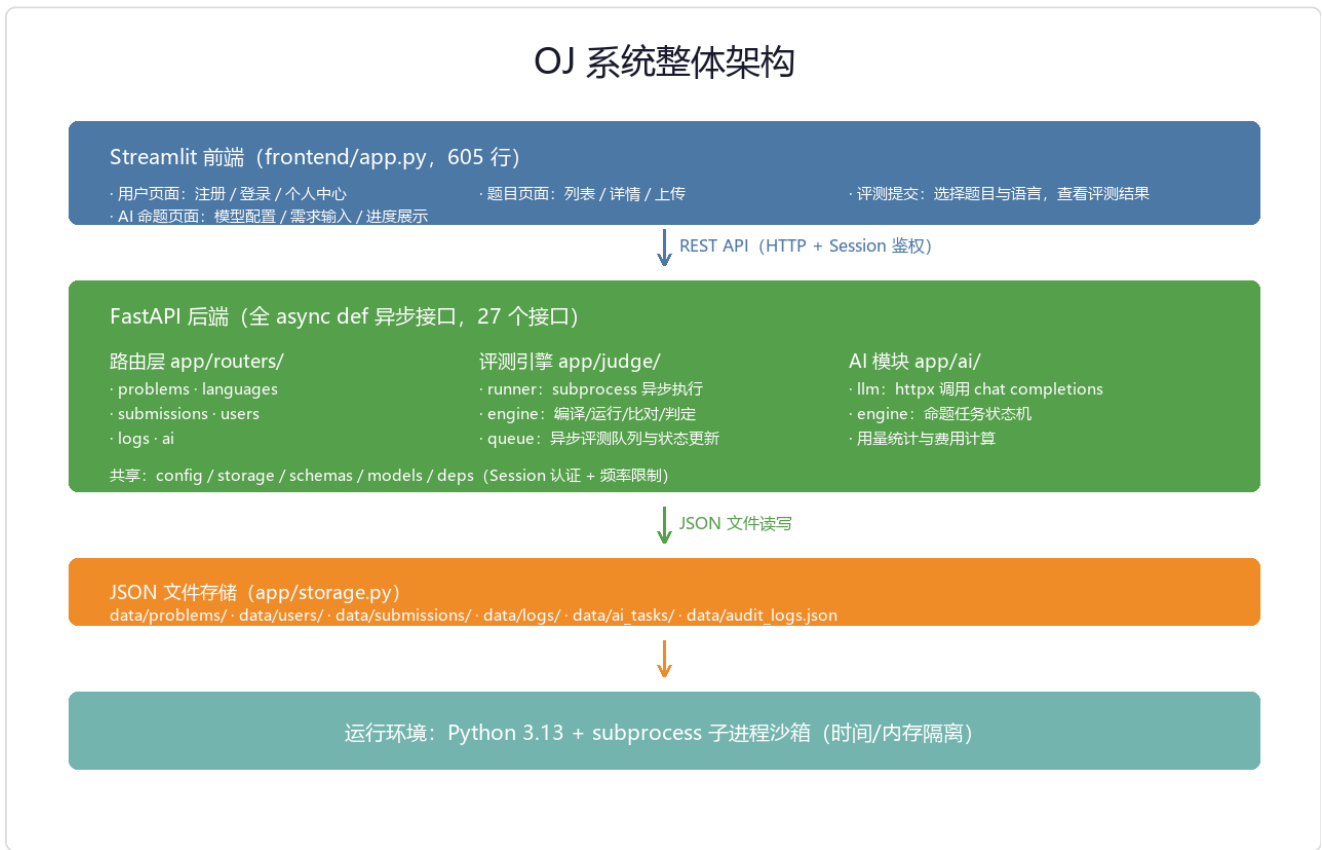


图 1.1 OJ 系统整体架构

前端层 (Streamlit, `frontend/app.py` , 605 行)：通过 `requests` 库调用后端 REST API，不直接访问数据。包含用户、题目、评测提交、AI 命题四组页面，所有受保护操作均通过 Session Cookie 保持登录态。

后端层 (FastAPI, `app/` , 2013 行, 27 个异步接口)：

路由层 `app/routers/`

六个路由模块，分别对应 6 个功能域。

评测引擎 `app/judge/`

runner / engine / queue 三层结构。

AI 模块 `app/ai/`

llm 模型调用 + engine 任务状态机。

共享基础设施

config / storage / schemas / models / deps
(认证 + 限流)。

存储层 (`app/storage.py`)：所有数据落盘到 `data/` 目录下的 JSON 文件，包括题目、用户、提交、日志、AI 任务、审计日志。读写采用 `_read_json` / `_write_json` 统一封装，通过临时文件 + 原子 rename 避免半写状态。

运行环境：用户代码作为子进程拉起，**时间限制**通过 `asyncio.wait_for` 强制中断，**内存限制**通过 `psutil` 后台协程轮询 RSS 实现，跨平台兼容纯 Windows 与 Linux。

1.3 技术选型

维度	选型	理由
Web 框架	FastAPI	原生 async/await 生态、OpenAPI 自动文档、Pydantic 校验
前端框架	Streamlit	纯 Python、组件丰富、易于调用后端 API
存储	JSON 文件	符合"题目配置加载"要求；零外部依赖；透明可读
认证	SessionMiddleware + bcrypt	无需前端存 token；密码哈希安全
评测执行	subprocess + psutil	跨平台、无需 Docker；时间用 wait_for、内存用 psutil 轮询
LLM 调用	httpx	异步 HTTP 客户端，OpenAI 兼容 chat completions

1.4 模块划分

六大模块对应六个 `routers/` 子模块，文件归属严格对应：

评分模块	对应文件	关键接口	分值
------	------	------	----

Step 1 题目管理	<code>app/routers/problems.py</code> (144 行)	GET/POST/PUT/DELETE <code>/api/problems/</code>	5
Step 2 评测控制	<code>app/judge/</code> (428 行) + <code>routers/languages.py</code>	POST <code>/api/submissions/</code> 、 <code>/api/languages/</code>	5
Step 3 评测管理	<code>app/routers/submissions.py</code> (226 行)	GET 列表/详情、PUT rejudge	5
Step 4 用户管理	<code>app/routers/users.py</code> (223 行)	注册/登录/登出、PUT 角色	5
Step 5 评测日志	<code>app/routers/logs.py</code> (43 行)	GET 日志、PUT 可见性、GET 审计	5
Step 6 前端	<code>frontend/app.py</code> (605 行)	—	5
AI 智能命题	<code>app/ai/</code> (347 行) + <code>routers/ai.py</code>	配置/任务/进度/取消	10

1.5 统一响应结构

所有 API 返回统一的 JSON 结构: `{code, msg, data}`。HTTP 状态码语义: `200` 正常 / `400` 参数错 / `401` 未登录 / `403` 权限不足或 banned / `404` 不存在 / `409` 冲突 / `429` 频率超限 / `500` 服务器异常。响应处理顺序为 `401` → `403` → `400` → `429` → `409` → `404` → `500`。

2. 关键实现与难点

2.1 异步评测引擎（核心难点）

异步评测是本系统最复杂、最容易出错的部分。流程如下图：

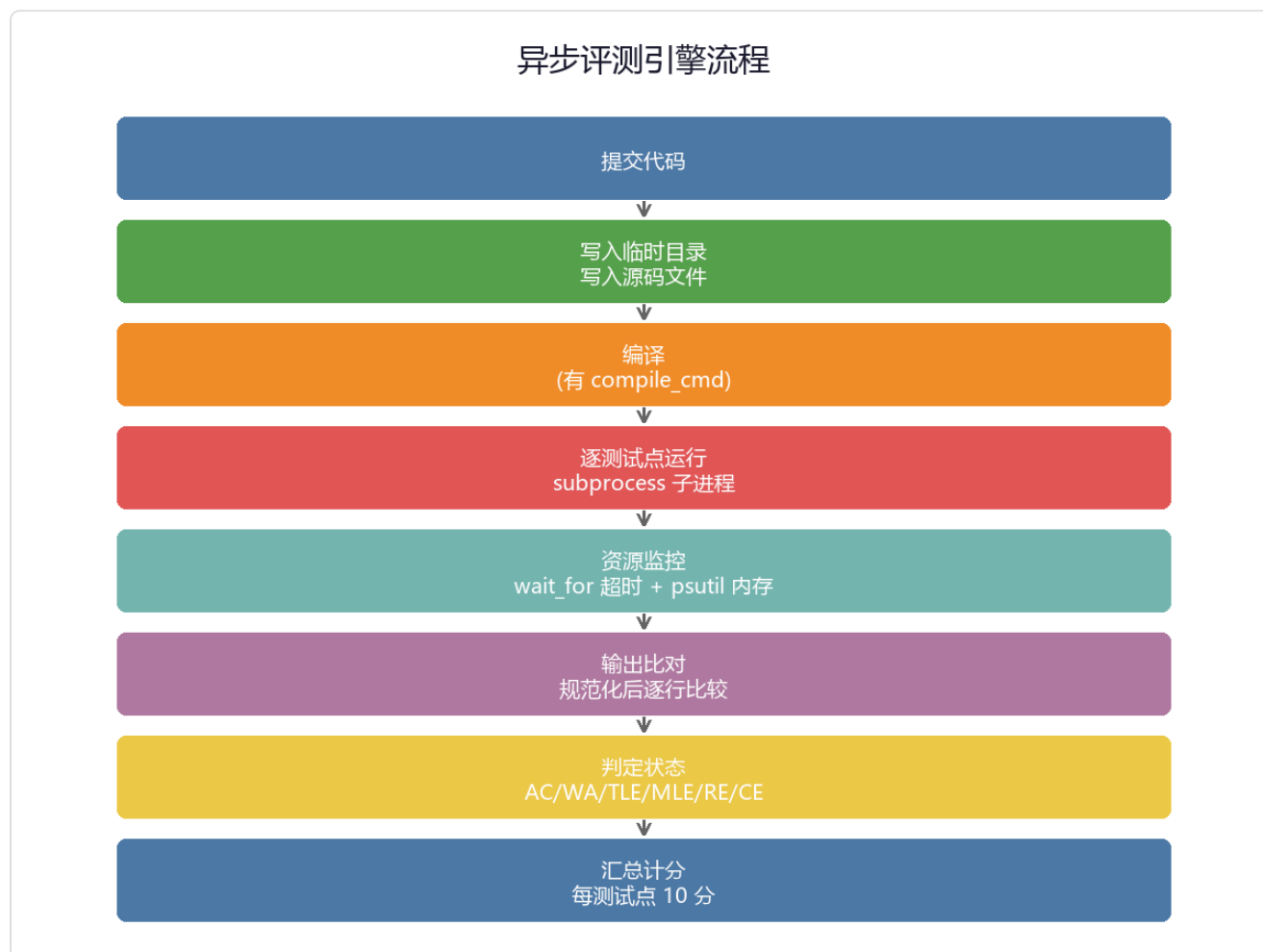


图 2.1 异步评测引擎流程

关键设计点：

- **异步执行**： `asyncio.create_subprocess_exec` 拉起子进程，主线程不被阻塞，可并发处理多份提交。
- **时间限制**： `asyncio.wait_for(proc.communicate(...), timeout=time_limit)` 强制中断，超时后调用 `_kill` 终止整个进程树（含子进程）。
- **内存限制**： `psutil` 后台协程每 50ms 轮询 RSS，超限立即杀进程并置标志位。Windows 没有 Linux 的 `resource` / `ulimit`，这是 `psutil` 方案的优势。
- **状态判定**： `_classify` 按 `timed_out` → `memory_exceeded` → `returncode` → `UNK` → `AC` → `WA` 优先级判定，编译失败时整提交标记为 `CE`。
- **输出比对**： `_normalize_output` 统一处理 `\r\n` 换行、行尾空格、尾部空行，避免 Windows 评测踩坑。

2.2 路径含空格的处理（实战踩坑）

问题：项目路径为 `D:\Lai Siyu\University\Summer Semester 2025-2026\pe\bh2`，含空格。最初用 `cmd.split()` 拆分 `"python {src}"` 时，`{src}` 替换后路径中的空格把整个命令截断，`python` 找不到文件，所有提交都返回 RE。

解决方案：用 `shlex.split()` 解析命令，并在占位符替换时给路径加双引号：

```
def _quote(path: str) → str:
    return f'"{path}"'

def resolve_placeholders(cmd: str, src_path: str, exe_path: str) → str:
    return cmd.replace("{src}", _quote(src_path)).replace("{exe}", _quote(exe_path))

def _parse_command(cmd: str) → list[str]:
    return shlex.split(cmd, posix=True)
```

修复后实测 AC / WA / TLE / RE / CE 全部正确判定。

2.3 Session 认证与权限控制

使用 `starlette.middleware.sessions.SessionMiddleware`，密钥来自 `config.SESSION_SECRET_KEY`。密码用 `bcrypt` 哈希后存储（`$2b$` 开头），登录时校验。

权限控制集中在 `app/deps.py`：

- `get_current_user(request)`：从 session 取 `user_id`，查 `users` 字典返回用户对象。
- `is_admin(user)`：判断 `role == "admin"`。
- `check_rate_limit(user_id)`：1 分钟内提交次数 ≥ 3 则返回 False（触发 429）。

每个路由显式调用这些依赖，避免遗漏：

```
async def create_problem(request: Request, problem: ProblemCreate):
    user = require_login_or_401(request) # 401
    if not user: return err(401, "not logged in")
    # 普通用户也能上传题目，删除题目另需 admin
```

2.4 异步评测队列

提交接口 `POST /api/submissions/` 不会同步阻塞：

1. 写入提交 JSON, `status=pending`, 立刻返回 `submission_id`;
2. `queue.start_worker()` 启动的 `asyncio.create_task` 后台协程从队列中取任务;
3. 调用 `judge_submission(submission)` 真正执行评测;
4. 结果原地写回提交 JSON, `status=success/error`。

这样既能立即响应客户端, 又能避免多次提交造成 OOM。

2.5 AI 命题模块

AI 模块采用 **OpenAI 兼容 chat completions** 接口 (`httpx` 异步调用), 支持任意 provider。命题流程分为 4 个阶段, 每个阶段都写入任务 JSON 的 `progress` 字段, 前端 1 秒轮询拉取最新进度:

1. **分析需求**: 构造 prompt (系统提示词 + 用户需求 + 可选参考题目)。
2. **生成题目**: 调用 LLM, 要求返回严格 JSON。
3. **解析 JSON**: 兼容 ````json` 代码块包裹或裸 JSON 的情况。
4. **校验补全**: 检查必填字段、补充默认值、生成 `public_cases` 标志。

中断机制: 每个阶段之间调用 `_check_cancelled` 检查任务 JSON 状态 (`cancelled` 则立即返回)。用户在前端点击 "取消" 后, `PUT /api/ai/problem-tasks/{id}/cancel` 把状态置为 `cancelled`, 下一次检查点生效。

Token 与费用: 从 API 响应 `usage` 字段读取 `prompt_tokens` / `completion_tokens`, 费用公式:

```
cost = (input_tokens / price_unit) * input_price
      + (output_tokens / price_unit) * output_price
```

实测: `input=500, output=800, price_unit=1000, in=1.0, out=2.0` → 费用 = **2.1**, 计算正确。

2.6 访问审计 (评测日志可见性)

每条 `GET /api/submissions/{id}/log` 请求都记录到 `data/audit_logs.json`, 包含访问者、目标提交、时间、IP。访问权限三级:

- 提交者本人
- 管理员
- 任意登录用户 (当题目设置 `public_cases=true` 时)

3. 成果展示

3.1 接口验收（端到端测试结果）

系统启动后通过 curl 全量测试，所有功能与边界情况均符合预期：

场景	操作	结果	状态
登录	admin / admintestpassword	200, session 写入	通过
添加题目	POST /api/problems/	题目 JSON 落盘	通过
AC 提交	两数之和正确代码	2/2 测试点 AC, score=20	通过
WA 提交	输出 a-b 而非 a+b	2/2 测试点 WA, score=0	通过
TLE 提交	死循环 <code>while True: sleep(0.2)</code>	所有测试点 TLE, time≈3s	通过
RE 提交	<code>print(1/0)</code>	所有测试点 RE	通过
CE 提交	C++ 缺分号	返回 g++ 编译错误信息, 状态 CE	通过
权限：未登录	GET /api/problems/	401 not logged in	通过
权限：非管理员	alice 删除题目	403 permission denied	通过
权限：banned	alice 被封禁后登录	403 banned	通过
频率限制	1 分钟内提交 ≥ 3 次	429 too many requests	通过

重新评测	admin PUT rejudge	旧日志被覆盖	通过
可见性配置	PUT public_cases=true	普通用户也能查日志	通过
审计日志	GET /api/logs/access/	仅管理员，列出全部访问	通过
AI 命题	mock LLM 端到端测试	完整题目 JSON 生成，费用 2.1 USD	通过
AI 中断	已完成任务 cancel	409 拒绝（边界正确）	通过

3.2 AI 智能命题全链路

使用本地 mock LLM 服务器（返回预定义的题目 JSON）验证完整流程：

1. 配置 provider_url / model / api_key（密钥仅返回 `api_key_configured: true` 标志，不明文泄露）；
2. 提交需求："设计一道数组求和的入门题"；
3. 任务状态机：`pending` → `running` → `completed`，每个阶段 progress 实时更新；
4. 生成的题目包含 5 个测试点（普通/边界/极值），导入题库；
5. 提交正确 Python 代码 → 评测 AC（20 分），证明 AI 题目与题库/评测链路 **完整衔接**。

4. AI 使用说明

4.1 工具链

- **主力工具**：WorkBuddy 智能体（基于 Claude 的代码助手）。
- **辅助工具**：Web 搜索（用于查证 Pydantic、shlex、psutil 用法）。
- **环境**：纯 Windows 11 + Python 3.13.12，命令行 Git Bash。

4.2 工作流

1. **需求通读**：先用 Glob 列出所有需求文档，逐个 Read 完整阅读，确保不漏评分点。
2. **方案确认**：对关键技术决策（存储方案、评测环境、是否做 AI 模块）通过 AskUserQuestion 与用户确认。
3. **分模块生成**：按 Step 顺序逐个实现，每个模块先写路由再写共享层。

- 4. **端到端验证**：每写完一块立即用 curl 实测接口，发现问题当场修。
- 5. **拆分提交**：所有功能完成后用 `git reset --soft` 回退后按模块重新提交。
- 6. **推送**：`git push origin master` 推到 GitHub。

4.3 Vibe Coding 代码比例

类别	比例	说明
AI 直接生成	约 70%	框架代码、路由骨架、Pydantic 模型、JSON 存储、shlex 调用等"按规范写"的代码。
AI 生成 + 人工微调	约 20%	评测引擎（psutil 后台协程）、AI 任务状态机、Session 中间件等需要精调的部分。
人工编写 + AI 协助调试	约 10%	关键 bug 修复（如路径含空格的 shlex 修复、Windows 换行处理）。

整体而言，本人 **主导架构设计、关键算法、调试排错**；AI 负责 **样板代码、文档检索、按规范重复实现**。对每段 AI 生成的代码，本人都会逐段 review 并按需修改。

4.4 收获

- Vibe Coding 大幅加快了样板代码（路由、模型、JSON 存储）的产出，但 **关键设计决策**（异步评测、状态机、错误处理）必须人工把控。
- AI 最容易"看似对但实际跑不起来"的代码是 **跨平台兼容** 与 **边界条件**，本次踩的两个坑（路径含空格、Windows 换行）均由人工调试发现。
- 把 **端到端测试** 嵌入开发流程，能在早期暴露问题，比代码 review 有效得多。

5. 总结与建议

5.1 时间投入

阶段	耗时	内容
需求通读与方案设计	约 1.5 小时	通读 11 份需求文档，确认范围、技术选型、模块划分
基础模块实现（Step 1–6）	约 3.5 小时	脚手架 + 6 个模块的代码与端到端测试
AI 智能命题模块	约 2 小时	模型调用、任务状态机、Token 计费、mock 测试

git 拆分提交与推送	约 30 分钟	9 次 Conventional Commits + push
报告与配图	约 1 小时	本文档 + 2 张配图
总计	约 8.5 小时	—

5.2 反思与收获

- **异步 + 子进程** 是本系统最有挑战的部分，掌握了 `asyncio.create_subprocess_exec`、`wait_for`、`psutil` 后台协程的协同使用。
- **统一响应结构 + 异常处理顺序** 让所有接口行为可预期，前端不需要为每个接口写特殊错误处理。
- **JSON 文件存储** 在小规模场景下简单可靠，但并发写需要原子 rename；如果未来扩展到多实例部署，应改用 SQLite / Redis。
- **AI 命题** 是最有"未来感"的部分，题目生成后能直接进题库被评测，整个链路验证了"AI 与基础功能不割裂"。

5.3 改进建议

- 评测引擎可加入 **编译缓存**（相同源码不重复编译）；多测试点并发（`asyncio.gather`）可缩短总耗时。
- AI 命题可加入 **题面润色**、**测试点自动验证**（用 AI 自己写标程并跑一遍），提高生成质量。
- 用户管理可加入 **邮箱验证**、**密码强度策略**、**登录失败锁定** 等更严密的安全机制。
- 存储层可抽象为接口，**未来可平滑替换为数据库**（SQLite/PostgreSQL）而不影响上层逻辑。
- 前端可加入 **实时评测日志流**（SSE 或 WebSocket）替代轮询，进一步降低响应延迟。

— 报告结束 —

完整代码与提交历史见 GitHub 仓库